



GROUP PROJECT REPORT

WIA1005 - DATA STRUCTURE

EMERGENCY MEDICAL DISPATCH SYSTEM

LECTURER NAME: DR. MOHAMED N. M. LUBANI

GROUP 7 (OCC 10)

GROUP MEMBERS:	MATRIX NUMBER
WONG E JEFF JAPHETH	23057181
NUR ARINAH ALIYAH BINTI NASFARAZI	24001616
ANG KAI XIN	23113456
NUR ANITH RUWAIDA BINTI AMDAN	24001645
NUR FARAH IRDINA BINTI MOHD RIZAM	24001443

LINK TO GITHUB REPO: https://github.com/Cactacean/EmergencyMedicalDispatch	
LINK TO DEMO VIDEO: https://drive.google.com/file/d/1b8TJ6WkoA_2cudI6vYqnKXXLK-bzLtp0/view?usp=drive_link	

1. Problem Statement and Introduction

During medical emergencies, dispatchers must make fast and accurate decisions when assigning ambulances to patients. Manual dispatching can waste valuable time because the dispatcher has to check ambulance availability, compare locations, and estimate routes. These delays are dangerous for serious cases such as heart attacks, fire injuries, or other life-threatening emergencies.

This project implements an Emergency Medical Dispatch System using core data structures and algorithms. The system receives emergency calls, prioritizes them based on medical severity, assigns the nearest available ambulance, and calculates the shortest route using graph-based pathfinding. The main goal is to reduce dispatch delay while ensuring that serious emergencies are handled before non-urgent cases.

The project combines three main data structure concepts: a Graph to represent the city road network, a Priority Queue to manage critical and serious calls, and a Regular Queue to manage non-urgent calls in First-In, First-Out order. The dispatcher integrates these components into one workflow.

The project scenario includes three emergency calls: a heart attack at Location A, a minor car accident at Location B, and a house fire with burns at Location C. Since the heart attack has Severity 1, it should be processed first. The house fire has Severity 2, so it should be processed next. The minor car accident has Severity 3 and should be processed only after urgent cases are cleared.

2. Graph Implementation and Dijkstra's Algorithm

The city road network is represented as a weighted, undirected graph. Each location, such as an ambulance base, intersection, or emergency site, is stored as a node. Each road between two locations is stored as an edge with a weight representing travel time in minutes.

The graph is stored using an adjacency list. This is more suitable than an adjacency matrix because road networks are usually sparse, meaning each location connects to only a few other locations. An adjacency list saves memory and allows the algorithm to check only the actual neighbours of a node.

```
private final Map<String, List<Edge>> adjacencyList;

public static class Edge {
    String destination; // adjacent location
    int weight;         // travel time in minutes
}
```

To assign an ambulance, the system must find the shortest route from each available ambulance to the emergency location. This is done using Dijkstra's algorithm. The algorithm repeatedly selects the unvisited node with the smallest known distance and relaxes its neighbouring edges. A priority queue is used inside Dijkstra's algorithm so that the next closest node can be selected efficiently.

The graph module returns both the estimated travel time and the shortest route. For example, if the fastest route from Base1 to Location D is Base1 -> A -> C -> D, the system can display both the route and the total travel time.

```
while (!minHeap.isEmpty()) {
    NodeDistance current = minHeap.poll(); // closest unsettled node
    String u = current.node;
    if (!visited.add(u)) continue;         // skip stale duplicate
    if (u.equals(target)) break;           // early termination

    for (Edge edge : adjacencyList.get(u)) { // relax neighbours
        String v = edge.destination;
        if (visited.contains(v)) continue;
        int newDist = distance.get(u) + edge.weight;
        if (newDist < distance.get(v)) {
            distance.put(v, newDist);
            predecessor.put(v, u);
            minHeap.offer(new NodeDistance(v, newDist));
        }
    }
}
```

Using a priority queue, the time complexity of Dijkstra's algorithm is $O((V + E) \log V)$, where V is the number of locations and E is the number of roads. This is efficient for the sample city and can also scale reasonably well for larger sparse road networks.

3. Priority Queue and Regular Queue Design

The system separates emergency calls into two queues. Severity 1 and Severity 2 calls are placed into the Priority Queue because they represent critical and serious emergencies. Lower severity numbers indicate higher urgency, so Severity 1 calls are processed before Severity 2 calls.

Severity 3 calls are placed into the Regular Queue. These are non-urgent cases, such as minor car accidents or minor injuries. The Regular Queue follows First-In, First-Out order, meaning the earliest minor call is processed first.

- Method

```

95 public EmergencyCall dequeue() {
96     if (isEmpty()) {
97         throw new IllegalStateException(
98             "Priority queue is empty. No call to dequeue.");
99     }
100     EmergencyCall highest = heap[1];
101     heap[1] = heap[size];
102     heap[size] = null; // help GC
103     size--;
104     if (!isEmpty()) {
105         siftDown(1);
106     }
107     return highest;
108 }

```

- Helper

```

148 private void siftDown(int i) {
149     while (true) {
150         int left = 2 * i;
151         int right = 2 * i + 1;
152         int highest = i;
153
154         if (left <= size && hasHigherPriority(heap[left], heap[highest]))
155             highest = left;
156         if (right <= size && hasHigherPriority(heap[right], heap[highest]))
157             highest = right;
158
159         if (highest != i) {
160             swap(i, highest);
161             i = highest;
162         } else {
163             break;

```

This two-queue design is safer than a normal FIFO-only system because a critical call should not wait behind minor cases. At the same time, non-urgent calls are still preserved in the Regular Queue and processed when the Priority Queue becomes empty.

For a heap-based priority queue, insertion and removal take $O(\log n)$ time. For the Regular Queue, enqueue and dequeue operations can be performed in $O(1)$ time when implemented with a linked queue.

4. System Workflow and Integration

The dispatcher is the central controller of the system. When a new emergency call is received, it first checks the severity. If the call has Severity 1 or Severity 2, it is inserted into the Priority Queue. If it has Severity 3, it is inserted into the Regular Queue.

```

public void receiveCall(EmergencyCall call) { 14 usages
    System.out.println("\nIncoming Call: " + call);

    // Severity 1 & 2 -> Priority Queue
    if (call.getSeverity() <= 2) {
        priorityQueue.offer(call);
        System.out.println("Added to PRIORITY queue.");
    }

    // Severity 3 -> Regular Queue
    else {
        regularQueue.offer(call);
        System.out.println("Added to REGULAR queue.");
    }

    processCalls();
}

```

After the call is queued, the dispatcher checks ambulance availability. If no ambulance is available, the call remains in its queue. If an ambulance is available, the dispatcher processes the Priority Queue first. Only when the Priority Queue is empty will the Regular Queue be processed.

```

public void processCalls() { 3 usages  Δ ANG KAI XIN
    while (hasAvailableAmbulance()) {
        EmergencyCall nextCall = null;

        // PRIORITY QUEUE FIRST
        if (!priorityQueue.isEmpty()) {
            nextCall = priorityQueue.poll();
            System.out.println("\nProcessing PRIORITY call: " + nextCall);
        }

        // THEN REGULAR FIFO QUEUE
        else if (!regularQueue.isEmpty()) {
            nextCall = regularQueue.poll();
            System.out.println("\nProcessing REGULAR call: " + nextCall);
        }

        // No calls remaining
        else {
            break;
        }

        assignAmbulance(nextCall);
    }
}

```

Once the next call is selected, the dispatcher gathers all available ambulances and sends their locations to the Graph module. The Graph module uses Dijkstra’s algorithm to calculate the shortest route from each available ambulance to the emergency location. The ambulance with the shortest estimated travel time is selected.

After the ambulance is assigned, it is marked as busy. When the mission is completed, it is marked as available again. In this simulation, the mission is completed immediately to demonstrate repeated queue processing and dispatch cycles.

```
private void assignAmbulance(EmergencyCall call) { 1 usage 2 ANG

    Map<String, String> availableAmbulances = new HashMap<>();

    for (Ambulance ambulance : ambulances) {
        if (ambulance.isAvailable()) {
            availableAmbulances.put(
                ambulance.getId(),
                ambulance.getCurrentLocation()
            );
        }
    }
}
```

The complete workflow is: incoming emergency call -> check severity -> enter Priority Queue or Regular Queue -> check available ambulance -> process Priority Queue first -> use Graph and Dijkstra’s algorithm to find the nearest ambulance -> dispatch ambulance -> mark ambulance as busy -> complete mission -> mark ambulance as available -> process the next waiting call.

5. Test Cases and Results

Five test scenarios were used to verify the correctness of the system. Scenario 1 tested priority ordering and Dijkstra routing using a heart attack, house fire, and minor car accident. The expected dispatch order was Heart Attack -> House Fire -> Minor Car Accident.

Scenario 2 tested the case where both ambulances are busy. The system should not discard calls; serious calls remain in the Priority Queue while minor calls remain in the Regular Queue. Scenario 3 tested what happens when an ambulance becomes available again. The system should check the Priority Queue before the Regular Queue.

Scenario 4 tested calls with the same severity. Equal-severity calls should follow arrival order if timestamp-based tie-breaking is used. Scenario 5 tested all non-urgent calls, where all Severity 3 cases should be processed by the Regular Queue in FIFO order.

Overall, the test cases confirmed that the system can classify calls, prioritize emergencies, assign ambulances, calculate routes, and continue processing waiting calls correctly.

6. Conclusion

The Emergency Medical Dispatch System demonstrates how data structures and algorithms can support ambulance dispatching. The Graph module models the road network and calculates shortest routes using Dijkstra’s algorithm. The Priority Queue ensures that critical and serious emergencies are handled before minor cases, while the Regular Queue preserves FIFO ordering for non-urgent calls.

By integrating these modules, the dispatcher can receive emergency calls, select the correct queue, find the nearest available ambulance, dispatch it through the shortest route, and continue with the next waiting call. This satisfies the main requirements of graph pathfinding, priority-based emergency handling, FIFO queue handling, system integration, and test demonstration.

7. Appendix: Output Demo

City Map - Ambulance Route

System Status

ID	Location	Status
AMB-01	B	AVAILABLE
AMB-02	C	AVAILABLE

Description	Location	Severity	Queue
Broken Leg	B	3	Regular Queue
Minor Bruising	D	3	Regular Queue
Twisted Ankle	C	3	Regular Queue

System Log

Dispatch Log

System has been reset.

SCENARIO 5: All Non-Urgent Calls

All severity 3 -> Regular Queue FIFO order.

INCOMING CALLS:

- Broken Leg at B (Severity 3 - MINOR) -> Regular Queue
- Minor Bruising at D (Severity 3 - MINOR) -> Regular Queue
- Twisted Ankle at C (Severity 3 - MINOR) -> Regular Queue

Selected Ambulance: AMB-01

Route: Base1 -> A -> B

Press 'Next Step' to watch the ambulance travel along the route!

Ambulance AMB-01 moving from Base1 to A...

Travel time: 4 minutes

Ambulance AMB-01 moving from A to B...

Travel time: 3 minutes

Ambulance AMB-01 moving from B (DESTINATION REACHED!)

Ambulance has arrived at the emergency scene!

Journey complete! Ambulance has already reached the destination.